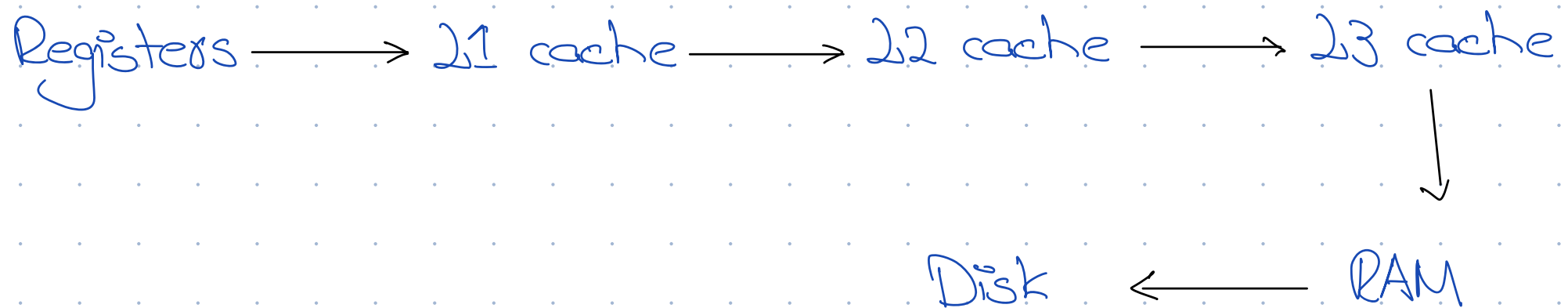


CPU's are very fast but RAM is slow.

Cache stores frequently used data.

Memory Hierarchy:

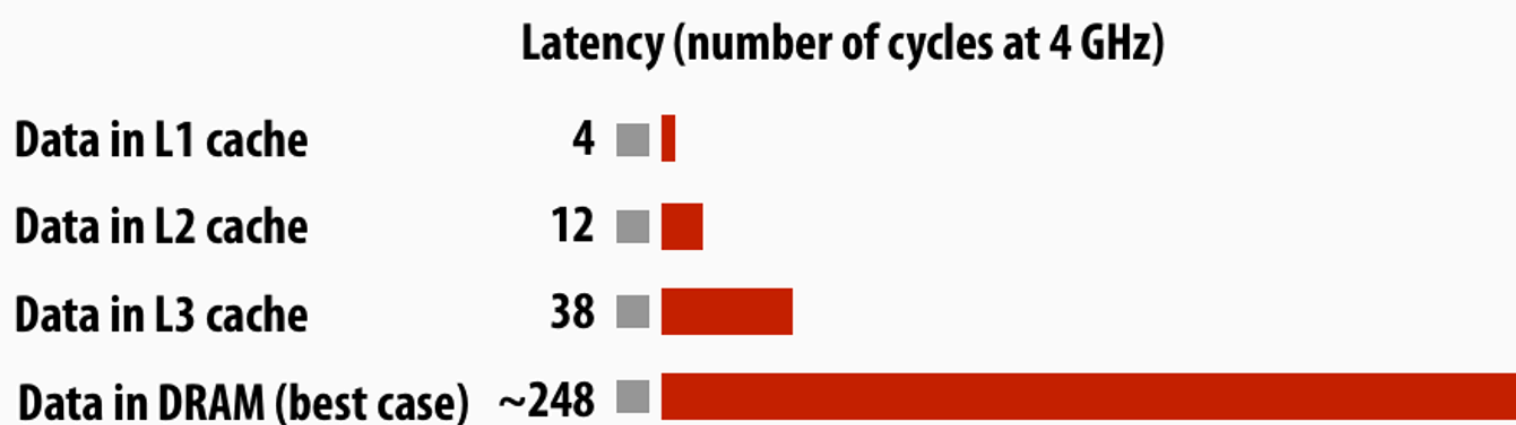


Cache principles:

- Temporal locality: Re-use of recent data
- Spatial locality: Nearby data likely accessed
- Cache stores blocks/lines of data

- **Temporal Locality:** This means that if you've recently used some data, you're likely to need it again soon. So, the cache keeps track of recently used information to access it quickly when needed again.
- **Example:** If you just opened a webpage, there's a good chance you'll visit that page again soon. So, the cache remembers it for faster loading next time.
- **Spatial Locality:** This refers to the idea that if you use one piece of data, there's a good chance you'll use nearby data soon after. So, the cache stores not just the exact data you just used but also the surrounding data.
- **Example:** If you're scrolling through a list of images, the cache doesn't just remember the one image you're looking at but also the ones near it, in case you scroll to them next.

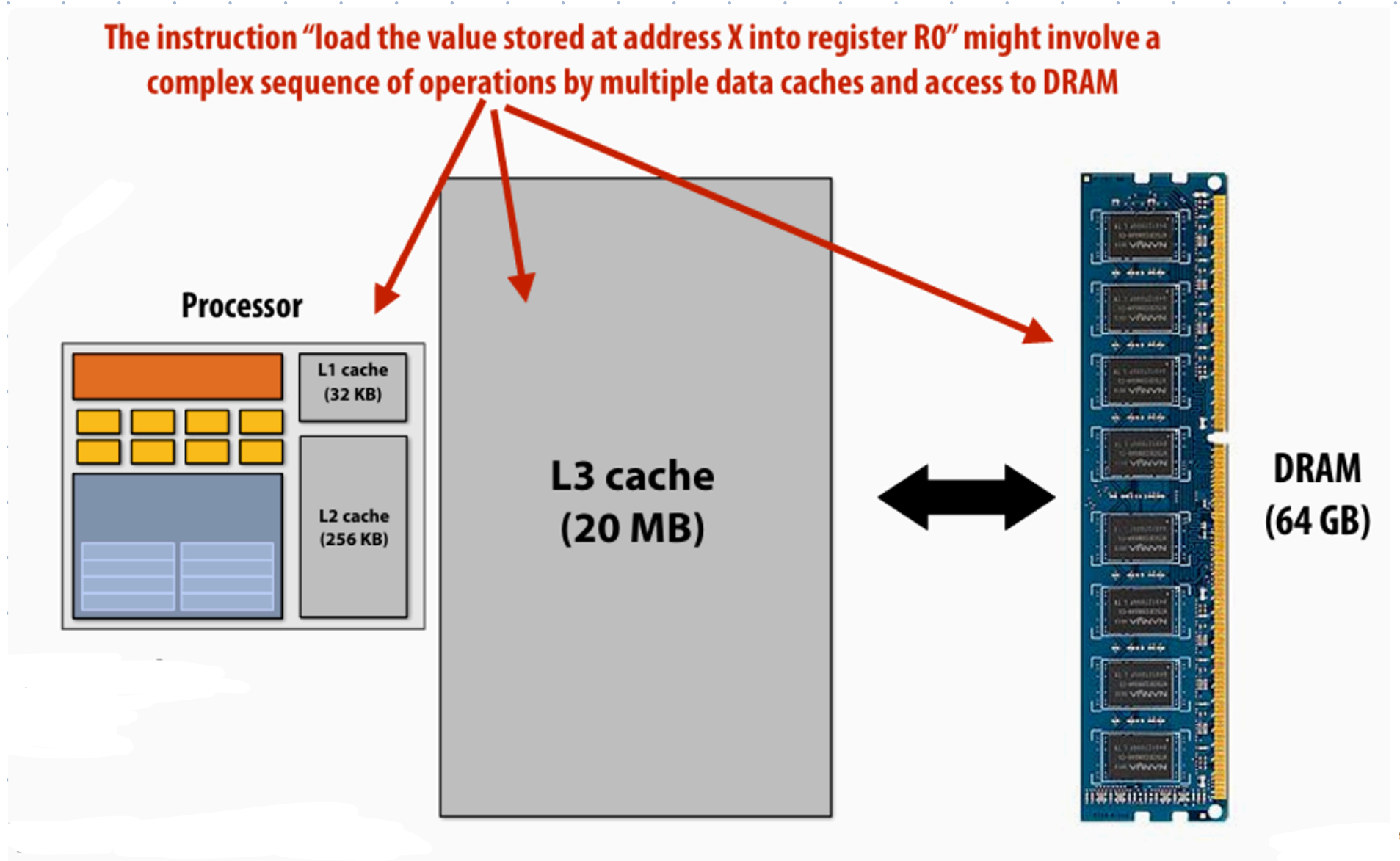
Data access times (if)



Caches reduce length of stall

↳ Processors run efficiently when they access data resident in caches.

↳ Caches reduce memory access latency.



What if data doesn't reside in caches?

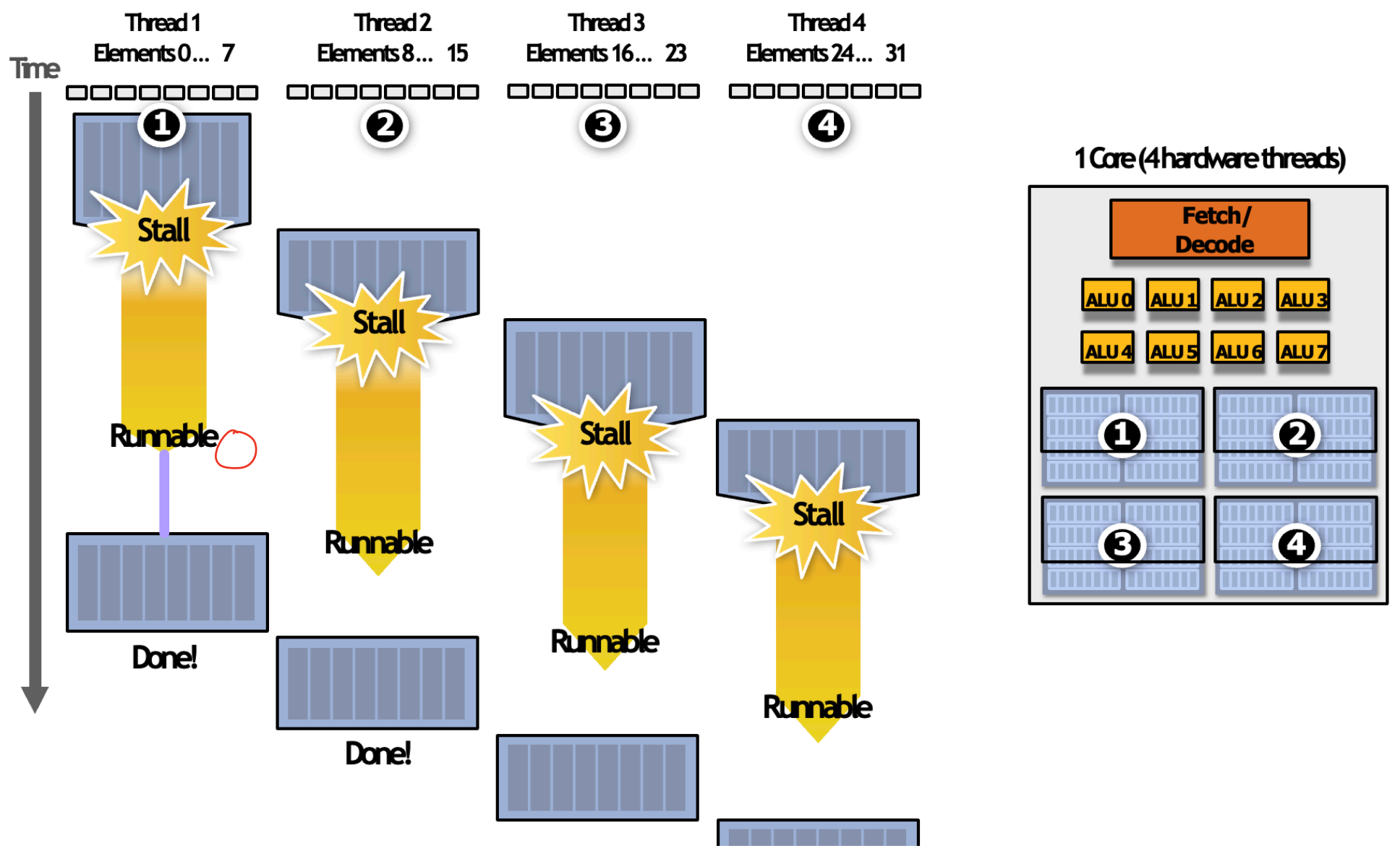
↳ A process looks for data first in L1 cache, if it does not find it there, it looks in L2 & then L3 & finally checks the RAM.

↳ Caches reduced the data access latency but in this case it's increased.

↳ This is referred to as a stall.

Multi-threading reduces stalls?

↳ Interleave processing of multiple threads on the same core to hide stalls.

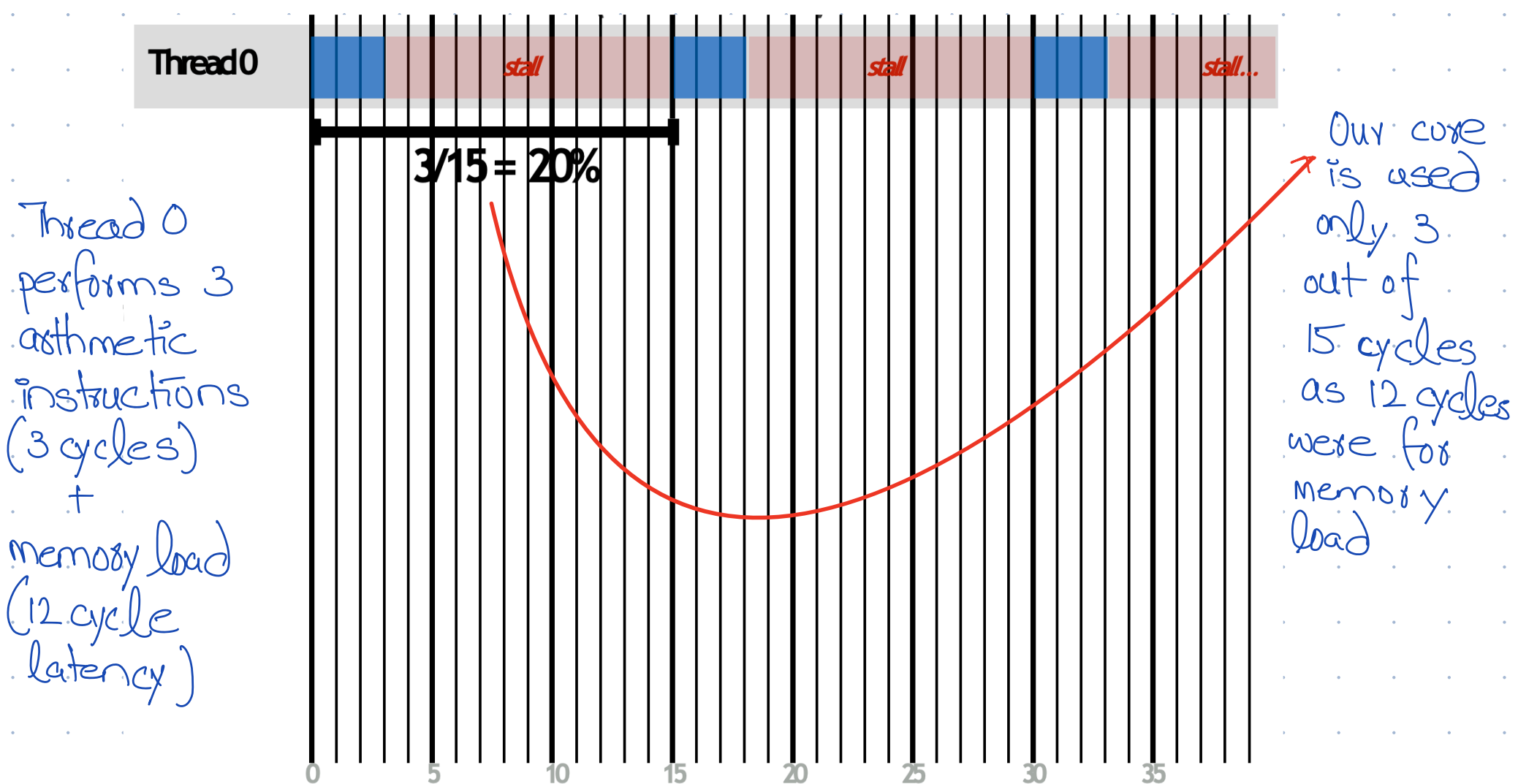


At this point, thread 1 is runnable but thread 4 is running. & it has to wait to run.

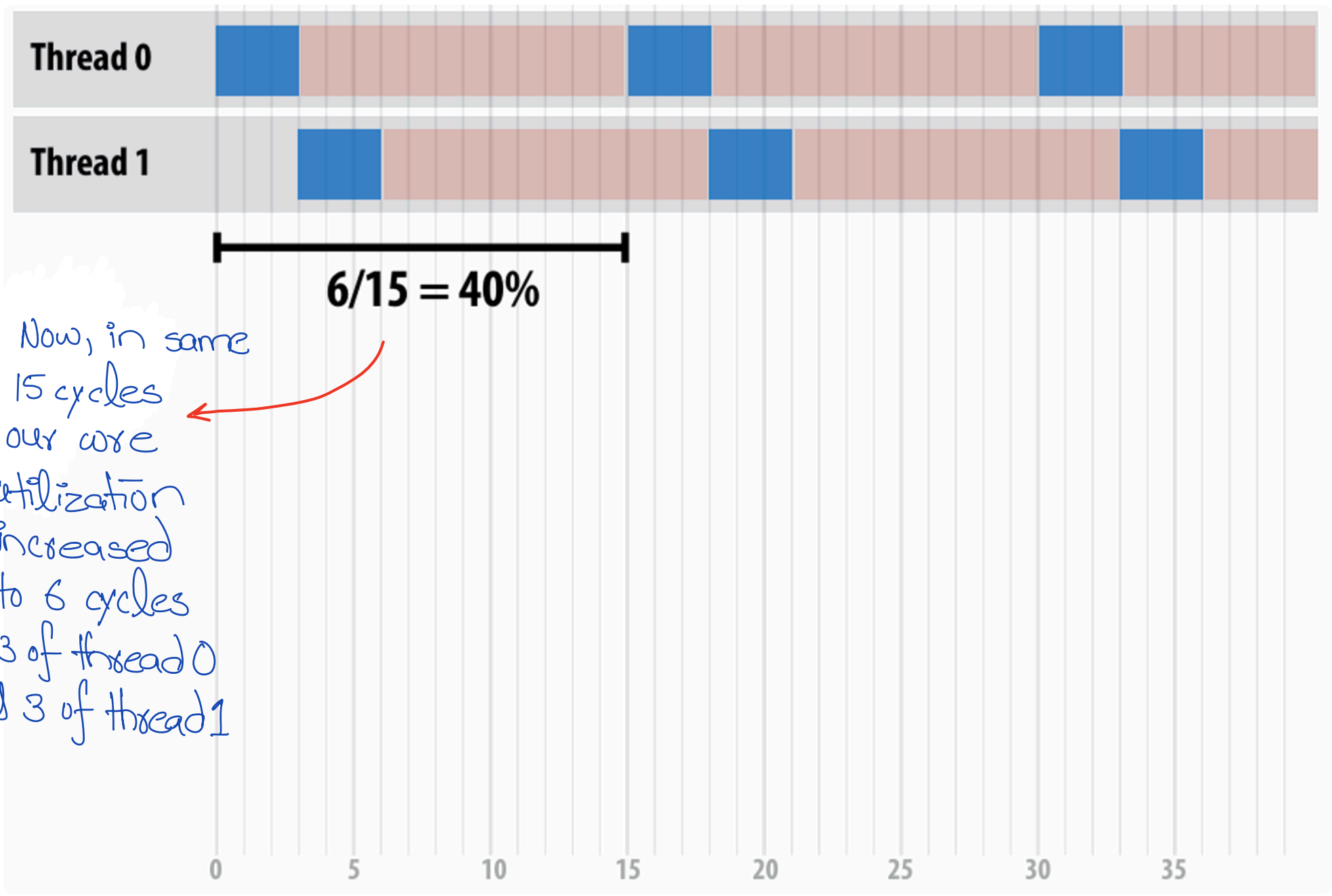
BUT this helps in the bigger picture.

The key idea is that we increase time to complete work by any one thread in order to increase overall system throughput when running multiple threads.

What is the utilization of the core? Week 06 Lec 01



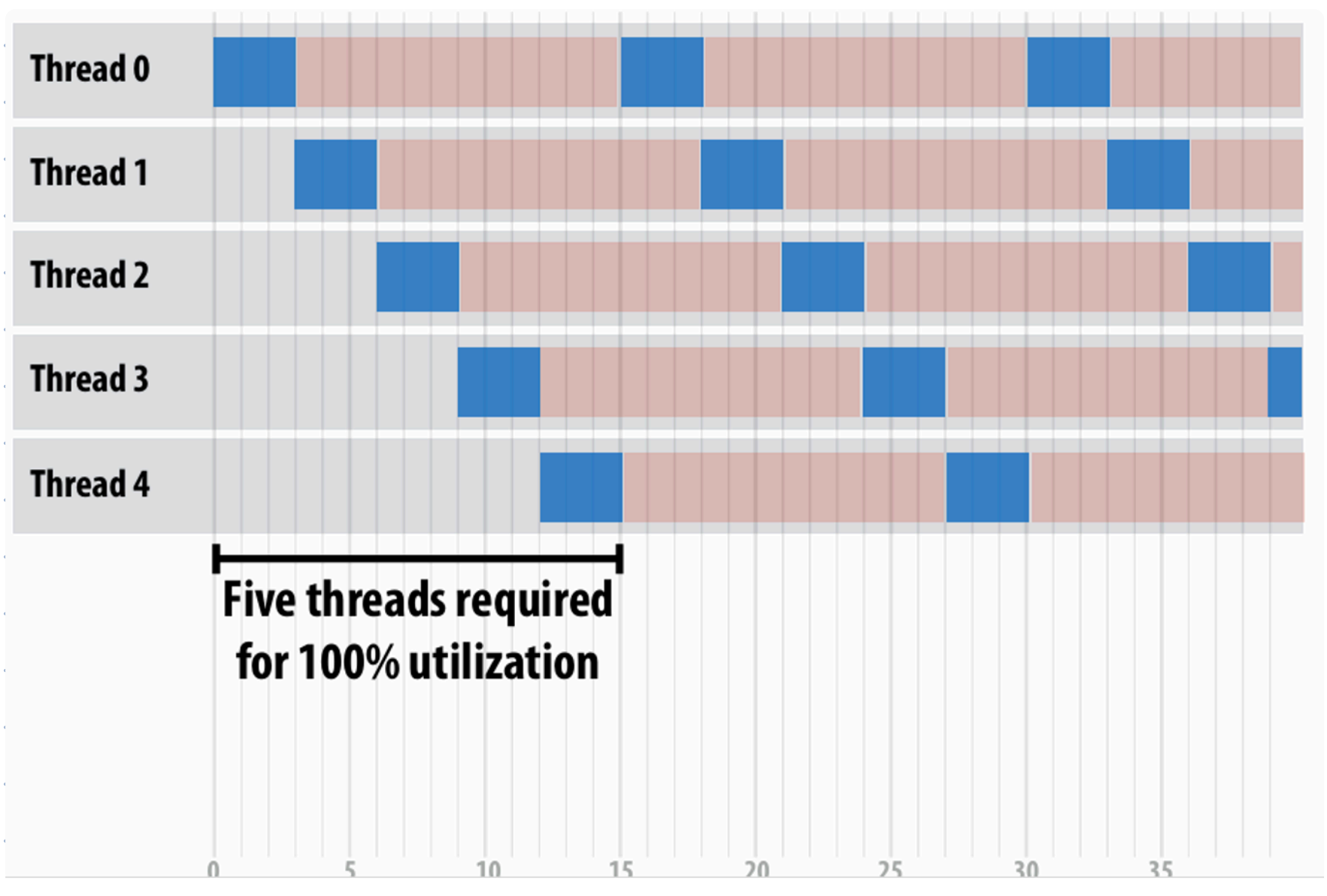
Lets add another thread 1, which has same behaviour as thread 0, when thread 0 does memory load.



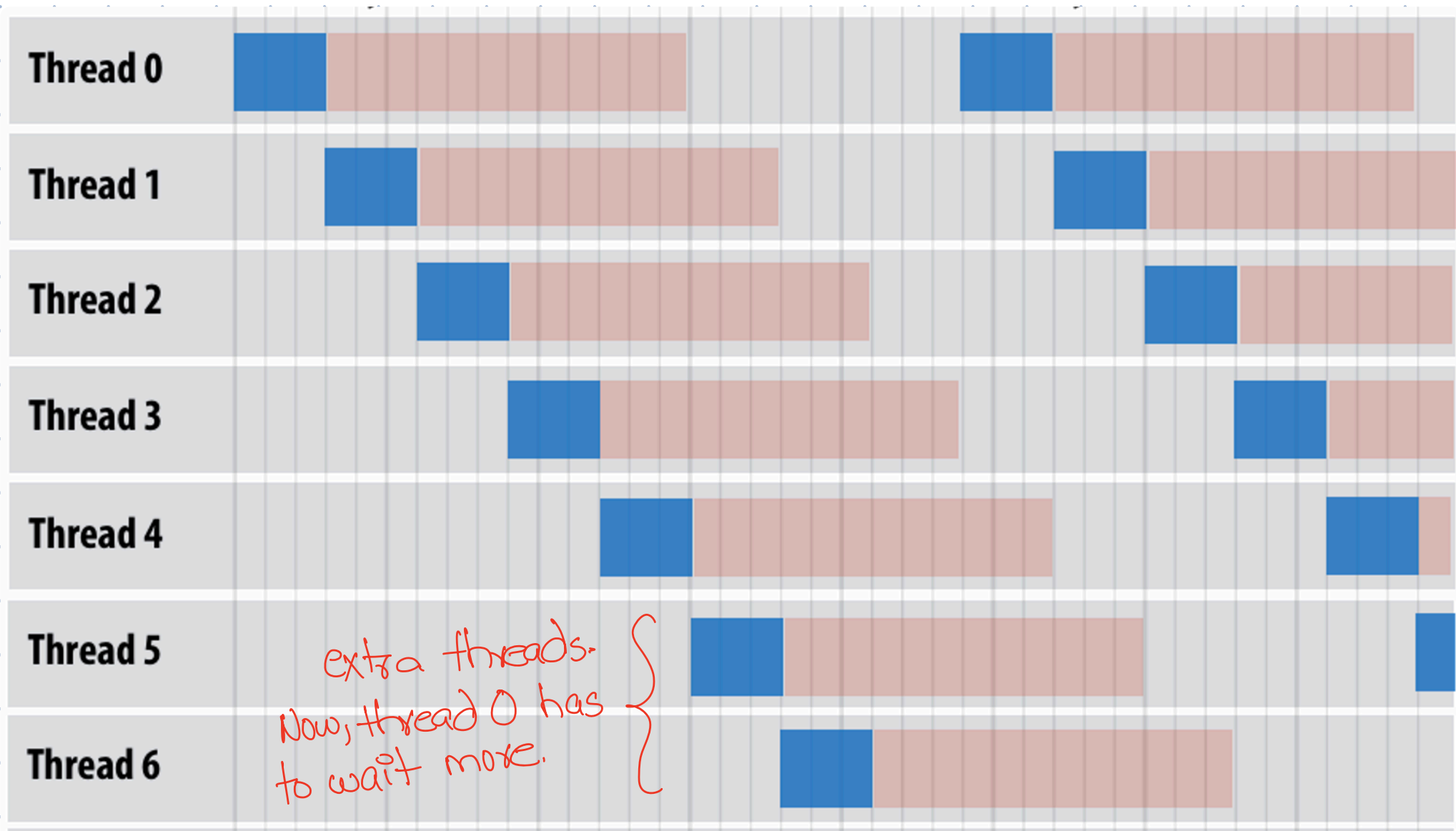
Now, in same 15 cycles our core utilization increased to 6 cycles
3 of thread 0
3 of thread 1

So, how many threads needed for 100% core utilization?

$$\frac{15 \text{ cycles}}{3 \text{ cycles per thread}} = 5 \text{ threads}$$



As we have reached 100% core utilization, adding more threads yield no benefit.



Now,
If threads perform 6 arithmetic operations + memory load (12 cycle latency), then how many threads are needed to achieve 100% core utilization?

$$6 + 12 = 18 \text{ total cycles}$$

$$\frac{18 \text{ cycles}}{\text{For 6 cycles, core is used}} = 3 \text{ threads}$$

Takeaways:

1. A processor with multiple hardware threads has the ability to avoid stalls by performing instructions from other threads when one thread must wait for a long latency operation to complete.

NOTE:

Latency of memory operation is not changed by multi-threading. It just no longer causes reduced

processor utilization.

2. A multi-threaded processor hides memory latency by performing arithmetic from other threads

Programs that feature more arithmetic per memory access need fewer threads to hide memory stalls.

